

Engine Dev — Bonnes pratiques

Cheat sheet de référence

Adam HAMOUC

May 2026

1. Mémoire & cache

- Données contigues → `vector<T>`, jamais `vector<T*>`.
- Stocker un **index**, jamais un pointeur/référence vers un élément de `vector` (réallocation = *danger*).
- `reserve()` si la taille est approximativement connue.
- `emplace_back()` > `push_back()` : construit directement, zéro copie.
- SoA (Structure of Arrays) > AoS pour les systèmes ECS : itération cache-friendly.

```
// SoA : iteraion cache-friendly
struct TransformStorage {
    vector<vec3> positions;
    vector<quat> rotations;
};
// Accés par index stable
uint32_t idx = entityToIndex[id];
transforms.positions[idx] = newPos;
```

2. Containers

- `vector` par défaut, toujours.
- `unordered_map` > `map` sauf si l'ordre est nécessaire.
- Clé numérique (`uint64_t`) > `string` : hash $O(1)$ vs $O(n)$.
- `RemoveSwap()` si l'ordre est sans importance → $O(1)$ vs $O(n)$.
- `unordered_set` pour les vérifications d'unicité fréquentes.

```
// Cle numerique precalculee
uint64_t key = hashString("PBR_Opaque");
pipelineCache[key] = pipeline;

// Suppression O(1)
swap(v[idx], v.back());
v.pop_back();
```

3. Ownership & smart pointers

- `unique_ptr` pour la propriété exclusive (composants, factory, Pimpl).
- `shared_ptr` pour les ressources partagées (textures, meshes).
- `weak_ptr` pour les liens enfant→parent, observers, caches.
- **Jamais** de raw pointer owning. Raw pointer = "je regarde, je ne possède pas".
- Parent→enfant : `unique_ptr`. Enfant→parent : `weak_ptr` (évite les cycles).

4. Design patterns clés

- **Factory / Abstract Factory** : backends de rendu (Vulkan/DX12), spawners d'entités.
- **Pimpl Idiom** : isoler les headers lourds (`vulkan.h`, `PhysX`) → temps de compilation.
- **ECS** : séparer données (composants) et logique (systèmes), SoA dense.
- **Observer** : événements moteur, `weak_ptr` sur les listeners.
- **RAII** : toute ressource (GPU buffer, mutex, fichier) wrapée dans un objet.

5. C++ moderne

- `auto` pour les itérateurs et types longs, `const auto&` pour ne pas copier.
- `nullptr` toujours, jamais 0 ou NULL.
- `()` par défaut pour l'init, `{}` seulement pour initialiser des containers avec valeurs.
- `static_cast` explicite plutôt que conversion implicite silencieuse.
- Préfer `make_unique` / `make_shared` → 1 alloc, ownership clair.

6. Rendu & GPU

- Minimiser les draw calls : batcher les meshes de même matériau.
- Tri front-to-back pour les opaques (depth prepass), back-to-front pour les transparents.
- Frustum culling avant tout upload GPU.
- UBO/SSBO : aligner les données (std140/std430), `alignas(16)` si nécessaire.
- Staging buffer → device local : ne jamais écrire directement en VRAM depuis le CPU.
- Barriers Vulkan explicites : ne pas laisser le driver deviner les dépendances.

7. Unreal Engine

- `TArray` = `vector`, `TMap` = `unordered_map`, `TSet` = `unordered_set`.
- `FName` comme clé de `TMap` (hash $O(1)$), jamais `FString`.
- `UPROPERTY()` sur tous les containers de `UObject*` → GC safety.
- `RemoveAtSwap()` si ordre sans importance → $O(1)$.
- `Emplace()` > `Add()` : construction directe, zéro copie intermédiaire.
- `FText` pour l'UI/localisation seulement, jamais comme clé ou logique interne.

8. Règles générales

- **Mesurer avant d'optimiser** : `RenderDoc`, `Nsight`, `Tracy`, `Unreal Insights`.
- **Scalable dès le départ** : penser en termes de N entités, pas 1.
- **Séparer interface et implémentation** : le client ne doit jamais dépendre des détails.
- **Une responsabilité par classe** : un système ECS ne fait qu'une chose.
- **Nommer explicitement** : `entityToIndex`, `pipelineCache`, pas `map1`.